Software Engineering Lab - Lab 7

Fuzzing for Input Validation Bugs

Arjun Srikanth | PES1UG24AM920 | Section H

Assigned program - Income Tax Parser

1. Objective

Use Hypothesis to test three input-processing functions, identify failures caused by unexpected inputs and correct the validation logic. The original and corrected versions were tested using the same test suite.

2. Findings and Corrections

Function	Issue in original program	Correction
clean_pan()	Non-string inputs caused attribute errors.	Check input type; normalize whitespace and case.
parse_amount()	Malformed amounts and missing values caused exceptions.	Validate format and grouping; return None for invalid input.
parse_tax_slab()	Incorrect delimiters or nonnumeric values caused exceptions.	Validate format, bounds and tax rate; return None for invalid input.

3. Initial Execution

The original script was executed with a sample PAN followed by an invalid rupee amount. The amount parser raised a ValueError and terminated the program.

```
Terminal

$ python income_tax_parser.py (sample PAN, invalid amount)
==== Running clean_pan ====
Enter a PAN (e.g. abcde 1234f): Cleaned PAN: ABCDE1234F

==== Running parse_amount ====
Enter an amount (e.g. Rs 12,50,000): Traceback (most recent call last):
  File "/workspace/se_labs_920_audit/repo/Lab-7/../../faculty/LAB7/income_tax_parser.py", line 39, in <module>
    main()
    ~~~~^^
  File "/workspace/se_labs_920_audit/repo/Lab-7/../../faculty/LAB7/income_tax_parser.py", line 32, in main
    print("Amount in rupees:", parse_amount(value))
                               ~~~~~~~~~~~~^^^^^^^
  File "/workspace/se_labs_920_audit/repo/Lab-7/../../faculty/LAB7/income_tax_parser.py", line 14, in parse_amount
    return int(text)
ValueError: invalid literal for int() with base 10: 'not money'
```

Figure 1. Original application output showing a failure on invalid amount input.

4. Fuzz Testing Before Correction

The Hypothesis tests include string and None inputs, additional non-string values and example-based validation cases. The original application failed 19 of the 29 tests.

```
● ● ●  Terminal

$ python -m pytest -q --tb=line test_income_tax_parser.py

FAILED test_income_tax_parser.py::test_clean_pan_no_crash - Failed: clean_pan...
FAILED test_income_tax_parser.py::test_parse_amount_no_crash - Failed: parse_...
FAILED test_income_tax_parser.py::test_parse_tax_slab_no_crash - Failed: pars...
FAILED test_income_tax_parser.py::test_non_string_types_never_crash - Excepti...
FAILED test_income_tax_parser.py::test_clean_pan_examples[  abcde\t1234f \n-ABCDE1234F]
FAILED test_income_tax_parser.py::test_clean_pan_examples[None-] - AttributeE...
FAILED test_income_tax_parser.py::test_parse_amount_examples[rs. 1,234-1234]
FAILED test_income_tax_parser.py::test_parse_amount_examples[\u20b9 2,50,000-250000]
FAILED test_income_tax_parser.py::test_parse_amount_examples[Rs -50-None] - A...
FAILED test_income_tax_parser.py::test_parse_amount_examples[1,2,3-None] - As...
FAILED test_income_tax_parser.py::test_parse_amount_examples[-None] - ValueEr...
FAILED test_income_tax_parser.py::test_parse_amount_examples[not money-None]
FAILED test_income_tax_parser.py::test_parse_amount_examples[None-None] - Att...
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[500000-250000:5-None]
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[100-200:101-None]
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[10-20:three-None]
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[100 - 200-None]
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[-None] - Value...
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[None-None] - A...
19 failed, 10 passed in 0.99s
```

Figure 2. Failure summary from the original program (19 failed, 10 passed).

5. Execution After Correction

The functions were updated to check input types and validate amount and slab formats before conversion. Invalid values no longer cause unhandled exceptions.

```
● ● ●  Terminal

$ python income_tax_parser.py (same inputs after validation fixes)
==== Running clean_pan ====
Enter a PAN (e.g. abcde 1234f): Cleaned PAN: ABCDE1234F

==== Running parse_amount ====
Enter an amount (e.g. Rs 12,50,000): Amount in rupees: Invalid amount

==== Running parse_tax_slab ====
Enter a tax slab as LOW-HIGH:RATE (e.g. 250000-500000:5): Tax slab (low, high, rate): Invalid tax slab
```

Figure 3. Corrected application output for the same invalid inputs.

6. Test Results After Correction

The full test suite was rerun without changing the test cases. All 29 tests passed.

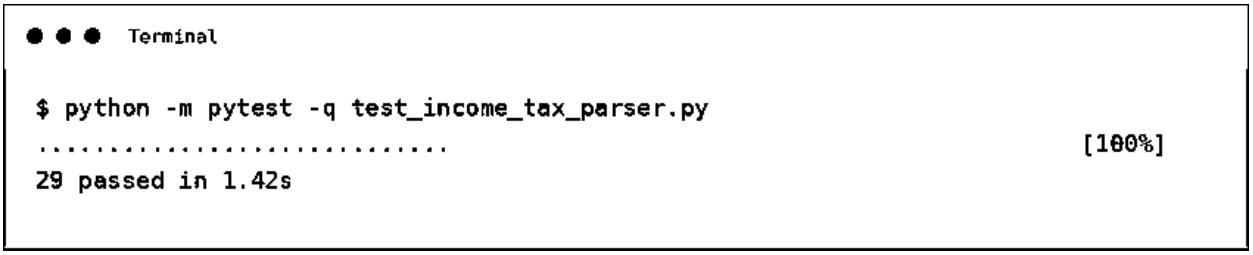


Figure 4. Final pytest result: 29 tests passed.

Check	Original program	Corrected program
Invalid amount input	ValueError	Handled without exception
Fuzzing and regression tests	19 failed, 10 passed	29 passed

7. Reflection Questions

1. What problems were identified during fuzzing?

Missing values, non-string inputs and malformed number formats caused exceptions. Further tests identified incorrect comma grouping and invalid tax slab limits.

2. How did fuzzing improve the test cases?

Hypothesis generated inputs beyond the fixed examples, including empty strings and unexpected data types. This helped check whether invalid inputs were handled consistently.

3. What changes prevented the failures?

Input types and formats are checked before string operations and integer conversion. Invalid amounts and slabs return None; an invalid PAN input returns an empty string.

8. Result

All three functions handle the tested invalid inputs without crashing. The corrected program passes all 29 tests.